practical 1 for exam

In [1]:
```
import pandas as pd
import numpy as np
```

Explanation: pandas: Used for data manipulation and analysis. numpy: Used for numerical operations.

These libraries are essential for handling datasets efficiently.

2. Dataset Description & Source

We are using the Titanic Dataset, a popular dataset from Kaggle.

🔗 Source: Titanic Dataset (Kaggle)

✅ Description:

The dataset contains information about passengers aboard the Titanic, including:

Demographics (age, gender) Ticket class Fare Survival status

It is commonly used for data analysis and machine learning tasks

3. Load Dataset into Pandas DataFrame

In [5]:
```
#load dataset
df=pd.read_csv('tiatanic.csv')
df
```

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	7
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	5
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	
...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	1
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	3
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	3
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	

891 rows × 12 columns

In [6]:
```
df.head()
```

Out[6]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	F
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0

In [7]:
```
df.tail()
```

Out[7]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Far
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7

4. Data Preprocessing

a) Check Missing Values

In [9]:
```
df.isnull().sum()
#isnull() checks missing values.
#sum() counts missing values column-wise.
```

Out[9]:
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

(b) Basic Statistics

In [10]:
```
df.describe()
```

Out[10]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.00
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.00
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.91
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.00
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329

Gives summary statistics like:

Mean Standard deviation Min / Max values

(c) Dataset Info

In [11]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Shows:

Column names Data types Non-null values

(d) Dimensions of Dataset

In [14]:
```
#return (row,column) of dataset
df.shape
```

Out[14]:
```
(891, 12)
```

e) Variable Description (Important for Viva)

(Column Description PassengerId Unique ID Survived 0 = No, 1 = Yes Pclass Ticket class Name Passenger name Sex Gender Age Age SibSp Siblings/spouse count Parch Parents/children count Fare Ticket price Embarked Boarding port

5. Data Formatting & Normalization

(a) Check Data Types

In [16]:
```
df.dtypes
```

Out[16]:
```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

In [27]:
```
#single column data type

df['Age'].dtype
print(df['Age'].head(5))
```

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: Age, dtype: float64
```

(b) Convert Data Types (if required)

In [30]:
```
df['Age'] = df['Age'].astype(float)
df['Survived'] = df['Survived'].astype(int)
#astype() is used to change the data type of a column
#astype() is only used when data type is wrong

print(df['Survived'].head())
```

```
0    0
1    1
2    1
3    1
4    0
Name: Survived, dtype: int64
```

(c) Normalize Data (Example: Fare)

In [46]:
```
df['Fare'] = (df['Fare'] - df['Fare'].min()) / (df['Fare'].max() - df['Fare'].mi


#formula x(norm)=(x)-(xmin)/(xmax)-(xmin) It converts values into a range betwee
```

In [47]:
```
print(df['Fare'])
```

```
0      0.014151
1      0.139136
2      0.015469
3      0.103644
4      0.015713
         ...
886    0.025374
887    0.058556
888    0.045771
889    0.058556
890    0.015127
Name: Fare, Length: 891, dtype: float64
```

6. Convert Categorical Variables to Quantitative

(a) Using Label Encoding

In [48]:
```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
#map() replaces each value in a column
```

In [49]:
```
print(df['Sex'].head())
```

```
0   NaN
1   NaN
2   NaN
3   NaN
4   NaN
Name: Sex, dtype: float64
```

(b) Using One-Hot Encoding

In [50]:
```
#df = pd.get_dummies(df, columns=['Embarked'])
df.info()


#pd.get_dummies():

#Removes original column
#Creates new numeric columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 14 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          0 non-null      float64
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked_C   891 non-null    bool
 12  Embarked_Q   891 non-null    bool
 13  Embarked_S   891 non-null    bool
dtypes: bool(3), float64(3), int64(5), object(3)
memory usage: 79.3+ KB
```

In [ ]:

In [ ]: